

API for Controller

- LLM params: model used, temperature, max tokens, api key, org, ...
- LLM cost features: prompt token cost, response token cost, ...
- Instances of Prompter + Parser + Graph of Operations,
- Any additional input parameters (e.g., numbers to be sorted).

Available operations when building the GoO (extensible)

- Generate, Aggregate, Score, ... //see Prompter API
- KeepBest(N) //preserves N best scoring thoughts
- Repeat(k) //Repeat a given operation k times, generating k thoughts.
//For example, this enables "Aggregate" to generate multiple outcomes //of the combination operation. Each such thought is maintained //within the Graph Reasoning State and scored individually.

API for Prompter (extensible)

- Generate(t, k) //generate a prompt for k new thoughts, using thought t
- ValidateAndImprove(t) //generate a prompt to enhance thought t
- Aggregate(t1, ..., tk) //generate a prompt to combine thoughts t1, ..., tk
- Score(t) //score thought t
- Validate(t) //generate a prompt to validate the correctness of thought t

API for Parser (extensible)

ParseGenerate, ParseImprove, ParseScore, ParseAggregate, ParseValidate, ...

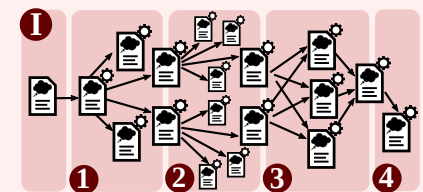
//Each of the above routines is responsible for parsing an LLM thought //to a corresponding Prompter routine (e.g., ParseScore parses Score).

Example prompts and the Graph Reasoning State for the sorting use case

(some examples within each prompt are omitted due to space constraints)

Initial/system prompt (optional)

Hello. I want to sort the following input sequence of numbers: {input}



A prompt used by Generate(t, k=4)

<Instruction> Split the following list of 64 numbers into 4 lists of 16 numbers each, the first list should contain the first 16 numbers, the second list the second 16 numbers, the third list the third 16 numbers and the fourth list the fourth 16 numbers. Only output the final 4 lists in the following format without any additional text or thoughts!

```
{
  "List 1": [3, 4, 3, 5, 7, 8, 1, ...,],
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...,],
  "List 3": [6, 9, 8, 1, 9, 2, 4, ...,],
  "List 4": [9, 0, 7, 6, 5, 6, 6, ...]
}
```

<Example>

Input: [3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0, 9, 7, 2, 2, 4, 4, 8, 5, 0, 8, 7, 3, 3, 8, 7, 0, 9, 5, 1, 6, 7, 6, 8, 9, 0, 3, 0, 6, 3, 4, 8, 0, 6, 9, 8, 4, 1, 2, 9, 0, 4, 8, 8, 9, 9, 8, 5, 9]

Output:

```
{
  "List 1": [3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0],
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...,],
  "List 3": [6, 9, 8, 1, 9, 2, 4, ...,],
  "List 4": [9, 0, 7, 6, 5, 6, 6, ...]
}
```

<Example>

Input: {input}

This prompt is used by an operation Generate where the branching factor is k=4. Four new thoughts are constructed based on the LLM reply to this prompt.

A prompt used by Generate(t, k=1)+Repeat(k=4)

<Instruction> Sort the following list of numbers in ascending order. Output only the sorted list of numbers, no additional text. **<Instruction>**

<Example>

Input: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9, 4, 3, 5, 6, 6, 4, 4, 5, 2, 0, 9, 3, 3, 9, 2, 1]
Output: [0, 0, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9]
<Example>

Input: {input}

This prompt is used by an operation Generate where the branching factor is k=1, which means, only one thought is generated. However, as we chain it with the operation Repeat with k=4, the underlying GoT framework ensures that Generate executes 4 times and results in 4 separate thoughts. Note that, from the graph theory perspective, the GRS is identical to that in the operation Generate(t, k=4).

The difference between these two is that Generate(t, k=4) gives the user more control over how these multiple thoughts are constructed, while Generate(t, k=1)+Repeat(k=4) is less flexible but more easy to use. Moreover, with Repeat one has 4 context-isolated responses from the LLM for identical prompts, whereas without Repeat there is only one context where all 4 thoughts are generated and must be explicitly handled in a single prompt/session.

A prompt used by Aggregate(t1, t2)+Repeat(k=3)+KeepBest(N=1)

<Instruction> Merge the following 2 sorted lists of length {length1} each, into one sorted list of length {length2} using a merge sort style approach. Only output the final merged list without any additional text or thoughts!

<Instruction>

<Approach>

To merge the two lists in a merge-sort style approach, follow these steps:

1. Compare the first element of both lists.

2. Append the smaller element to the merged list and move to the next element in the list from which the smaller element came.

3. Repeat steps 1 and 2 until one of the lists is empty.

4. Append the remaining elements of the non-empty list to the merged list.

<Approach>

Merge the following two lists into one sorted list:

1: {input1}
2: {input2}

Merged list:

This prompt is used by an operation Aggregate where the aggregation factor is k=2 (2 input thoughts, t1 and t2, are aggregated). This is repeated by GoT 3 times, to maximize quality. Finally, the best result is selected. Note that, in this example, the prompt explicitly requests the merge operation only. All the remaining operations are specified in the GoO and are handled by the underlying GoT framework.

Goal: Initiate, coordinate, manage, and progress the GoT execution

User

Controller

Goal: Specify LLM thought transformations

Graph of Operations

Goal: Build a prompt to be sent to the LLM

LLM

Prompter

Parser

Goal: Extract information from LLM thought

Human or LLM

Scoring & validation

Goal: Assess the quality of the LLM's solution

Graph Reasoning State

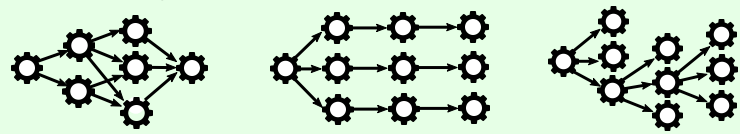
Goal: Maintain the ongoing LLM reasoning process

Ranking

Goal: Indicate the top-scoring thoughts

Specifying the Structure of the Graph of Operations (GoO)

Graph of Operations enables seamless specification of not only GoT, but also existing schemes such as CoT, CoT-SC, ToT



A prompt used by Improve(t)+Repeat(k=4)

<Instruction> The following two lists represent an unsorted list of numbers and a sorted variant of that list. The sorted variant is not correct. Fix the sorted variant so that it is correct. Make sure that the output list is sorted in ascending order, has the same number of elements as the input list (length), and contains the same elements as the input list. **<Instruction>**

<Approach>

To fix the incorrectly sorted list follow these steps:

1. For each number from 0 to 9, compare the frequency of that number in the incorrectly sorted list to the frequency of that number in the input list.

2. Iterate through the incorrectly sorted list and add or remove numbers as needed to make the frequency of each number in the incorrectly sorted list match the frequency of that number in the input list.

<Approach>

<Examples>

Input: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9]
Incorrectly Sorted: [0, 0, 0, 0, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 7, 7, 8, 8, 9, 9, 9, 9]
Reason: The incorrectly sorted list contains four extra 0s, two extra 4s and three extra 9s and is missing two 2s.

Output: [0, 1, 2, 2, 2, 3, 3, 4, 5, 5, 5, 7, 7, 8, 8, 9]

Input: [6, 4, 5, 7, 5, 6, 9, 7, 6, 9, 4, 6, 9, 8, 1, 9, 2, 4, 9, 0, 7, 6, 5, 6, 6, 2, 8, 3, 9, 5, 6, 1]

Incorrectly Sorted: [0, 1, 1, 2, 2, 3, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9, 9]

Reason: The incorrectly sorted list contains two extra 4s and is missing two 6s and one 9.

Output: [0, 1, 1, 2, 2, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9, 9, 9, 9]

<Examples>

Input: {input}

Incorrectly Sorted: {incorrectly_sorted}

This prompt is used by an operation Improve(t), which enhances a given thought t using information provided in another thought.

Depending on how the Improve + Repeat operation is implemented by the user within GoT, it can either generate a number of new thoughts in GRS (the upper graph on the right), similar to Generate + Repeat, or may refine the same thought in GRS (the lower graph on the right), chaining k=4 refinement iterations together.